

DOSSIÊ DE ARQUITETURA E STATUS DO PROJETO - SMARTML ULTRA

Data de Atualização: 29 de Agosto de 2026
Fase Atual: Transição de Arquitetura (Monolito Apps Script -> Microserviços Python/FastAPI)
Responsável Técnico: Luiz Antônio Camargo Fernandes

1. Visão Geral da Arquitetura Definitiva

O sistema foi dividido em duas metades (desacoplamento), seguindo o padrão ouro de engenharia de software:

- **FRONTEND (A Vitrine):** Google Sheets + Google Apps Script (HTML/JS).
 - **Papel:** É uma "tela burra". Capta a digitação, tira foto do EAN via OCR, envia os dados para o servidor, recebe a resposta e desenha na tela e formata a mensagem para o WhatsApp. Não executa processamento ou cálculos contábeis complexos.
- **BACKEND (O Cérebro):** Python + FastAPI hospedado na nuvem (Render).
 - **Papel:** É o motor pesado. Recebe o nome do produto, usa inteligência NLP (AuditorIAUltra) para varrer o Mercado Livre burlando bloqueios, faz a matemática financeira exata do Buybox (Premium vs Clássico) e devolve um pacote de dados (JSON) fechado para a Vitrine.

2. Histórico de Alterações e Ajustes

Versão	Arquitetura	Problema / Motivo da Evolução
V1	Monolito Local (Google Apps Script)	A API pública do ML bloqueava buscas complexas. O sistema inventava preços (markup de 1.35x) gerando falsos positivos matemáticos (R\$ 5.000 vs R\$ 8.000 = preços de concorrentes divergentes para o mesmo

Versão	Arquitetura	Problema / Motivo da Evolução
		modelo).
V2	Polling Assíncrono (Aguardando Robô)	O Python via FastAPI atua como API REST Síncrona, não como um robô de background que varre planilhas. A tela ficava travada eternamente aguardando uma edição celular.
V3 (Atual)	Microserviço Síncrono (HTTP Client)	O Google Apps Script faz um disparo direto (POST) para a URL do Render. Resposta rápida. Interface ajustada com botões independentes para App Flutuante e Planilha.

3. Diagnóstico do Erro Atual

Referência: Aviso "O servidor não enviou os dados completos de lucro. Confirme o deploy do *main.py*".

A blindagem de segurança da Vitrine funcionou para impedir o erro "null". A conexão HTTP com a nuvem (Render) foi bem-sucedida, mas ocorreu uma **Quebra de Contrato JSON**.

- **A Causa:** A vitrine espera receber blocos de dados rotulados como "premium" e "classico". A nuvem, no entanto, ainda está executando a versão anterior da arquitetura Python (que retorna o formato do arquivo *motor.py* com listas genéricas de "opcoes" e "faixas").
- **O Motivo:** O comando de deploy (`git push`) ainda não finalizou a compilação no serviço Render, ou o arquivo *main.py*

novo não foi salvo/commitado corretamente na máquina local antes de ser enviado para a nuvem.

4. Próximos Passos (Plano de Ação)

1. Sincronizar o Contrato (Ação Imediata):

- Validar se o backend no Render completou o build com o novo main.py focado no Buybox.
- *Via de contingência:* Caso a alteração no código Python não seja desejada no momento, o Google Apps Script deve ser adaptado para extrair os dados lendo o formato original (AnaliseProduto) gerado pelo motor.py legado.

2. Criação do Robô do WhatsApp (Avanço Tecnológico):

- Desenvolver um serviço em Python para automatizar o disparo das aprovações comerciais para grupos ou contatos, eliminando o gargalo operacional do clique manual "Copiar WhatsApp".

3. Persistência em Banco de Dados SQLite (Escalabilidade):

- Migrar a base de registros de cotações para um banco de dados relacional (SQLite). O Google Sheets passará a ser utilizado exclusivamente como dashboard/relatório gerencial, protegendo a integridade histórica dos dados (evitando exclusões acidentais ou corrupção de fórmulas na planilha).

Documento gerado automaticamente pelo Arquiteto de Soluções IA.